

----- -----		
10.x.r.0/27	10.x.r.0 - 10.x.r.31	demosys-k8s-clusterName-nodes 9
Nodes		
varies	10.x.r.32 - 10.x.r.128	(reserved for future)
(future use)		
10.x.r.128/25	10.x.r.128 - 10.x.r.255	demosys-k8s-clusterName-services 127
Services		
10.x.{r+1}.0/24	10.x.{r+1}.0 - 10.x.{r+1}.255	demosys-k8s-clusterName-pods 144
Pods (16/node)		

This design allows for 127 clusters per /16 CIDR range.

Learn more in our Terraform configuration module.

Medium Cluster (Specific Service)

We're going to make an assumption that we want each Kubernetes cluster for a specific service fit within a /20 CIDR range.

We are going to reduce the number of pods per node down to 55. As a rule of thumb, there should be a maximum of 4 for every 1 vCPU or 1GB Memory (whichever is less). This should be adjusted based on the gcpmachinetype chosen. With the /20 CIDR range of a medium cluster, we have /21 available for pods (1,022 addresses). With a maximum of 30 nodes in this cluster, this allows up to 34 pods per node with the appropriately sized machine type. For n1-highcpu-8 with 8 vCPU and 7.2gb memory, it is recommended to use a value of 32.

CIDR Range /20	Usable Host Range	
----- -----		
10.x.0.0 /20	10.x.0.1 - 10.x.15.254	
10.x.16.0 /20	10.x.16.1 - 10.x.31.254	
10.x.32.0 /20	10.x.32.1 - 10.x.47.254	
10.x.48.0 /20	10.x.48.1 - 10.x.63.254	
10.x.64.0 /20	10.x.64.1 - 10.x.79.254	
10.x.80.0 /20	10.x.80.1 - 10.x.95.254	
10.x.96.0 /20	10.x.96.1 - 10.x.111.254	
10.x.112.0 /20	10.x.112.1 - 10.x.127.254	
10.x.128.0 /20	10.x.128.1 - 10.x.143.254	
10.x.144.0 /20	10.x.144.1 - 10.x.159.254	
10.x.160.0 /20	10.x.160.1 - 10.x.175.254	
10.x.176.0 /20	10.x.176.1 - 10.x.191.254	
10.x.192.0 /20	10.x.192.1 - 10.x.207.254	
10.x.208.0 /20	10.x.208.1 - 10.x.223.254	
10.x.224.0 /20	10.x.224.1 - 10.x.239.254	
10.x.240.0 /20	10.x.240.1 - 10.x.255.254	

This design allows for 16 clusters per /16 CIDR range.

For documentation purposes, we use characters a through p to represent the 16 possible values of the 3rd octet since the value varies depending on the /20 CIDR range.

CIDR Range	IP Range	Subnet Name	Capacity
-----	-----	-----	-----

10.x.a.0/24	10.x.a.0 - 10.x.a.255	demosys-k8s-clusterName-nodes	32 Nodes
varies	10.x.b.0 - 10.x.d.255	(reserved for future)	(future
use)			
10.x.e.0/22	10.x.e.1 - 10.x.h.254	demosys-k8s-clusterName-services	1,024
Services			
10.x.i.0/21	10.x.i.1 - 10.x.p.254	demosys-k8s-clusterName-pods	1,024 Pods

Learn more in our Terraform configuration module.

Large Cluster (Multi-Tenant/Production)

We're going to make an assumption that we want each multi-tenant Kubernetes cluster to fit within a /16 CIDR range.

We are going to reduce the number of pods per node down to 55. As a rule of thumb, there should be a maximum of 4 for every 1 vCPU or 1GB Memory (whichever is less). This should be adjusted based on the gcpmachinetype chosen. With the /16 CIDR range of a large cluster, we have /17 available for pods (14,080 addresses). With a maximum of 255 nodes in this cluster, this allows up to 55 pods per node with the appropriately sized machine type. For n1-highcpu-16 with 16 vCPU and 14.4gb memory, it is recommended to use a value of 55 (rounded down from 57.6).

For documentation purposes, we're going to replace the second octet of the IP addresses and CIDR ranges with c to represent a /16 cluster.

CIDR Range	IP Range	Subnet Name	Capacity
-----	-----	-----	-----

10.c.0.0/23	10.c.0.0 - 10.c.1.255	demosys-k8s-clusterName-nodes	255 Nodes
varies	10.c.2.0 - 10.c.63.255	(reserved for future)	(future
use)			
10.c.64.0/18	10.c.64.1 - 10.c.127.254	demosys-k8s-clusterName-services	16,384
Services			
10.c.128.0/17	10.c.128.1 - 10.c.255.254	demosys-k8s-clusterName-pods	14,080 Pods

Learn more in our Terraform configuration module.

Kubernetes Scalability Considerations

Google Kubernetes Engine IP allocation

This is an excerpt from the Google Kubernetes Engine documentation

- Each node has an IP address assigned from the cluster's Virtual Private Cloud (VPC) network. This node IP provides connectivity from control components like kube-proxy and the kubelet to the Kubernetes API server. This IP is the node's connection to the rest of the cluster.
 - Each Service has an IP address, called the ClusterIP, assigned from the cluster's VPC network. You can optionally customize the VPC network when you create the cluster.
 - Each node has a pool of IP addresses that GKE assigns Pods running on that node (a /24 CIDR block by default). You can optionally specify the range of IPs when you create the cluster. The Flexible Pod CIDR range feature allows you to reduce the size of the range for Pod IPs for nodes in a given node pool.
 - You can run a maximum of 110 Pods on a node with a /24 range, not 256 as you might expect. This provides a buffer so that Pods don't become unschedulable due to a transient lack of IP addresses in the Pod IP range for a given node. For ranges smaller than /24, roughly half as many Pods can be scheduled as IP addresses in the range.
- Each Pod has a single IP address assigned from the Pod CIDR range of its node. This IP address is shared by all containers running within the Pod, and connects them to other Pods running in the cluster.

tip GitLab Lesson Learned

Our biggest IP allocation challenge is related to Kubernetes clusters and nodes that consume large blocks of IP addresses. When you have dozens or hundreds of users utilizing Kubernetes, there are several design considerations that need to be taken into account to ensure that you only allocate the necessary IP address blocks (ex. the default node subnet is a /20 and a node consumes a /24 subnet).

If each user has their own cluster with a pool of 3 nodes, it's very easy to run out of IP's at scale. This is a lesson that we learned the hard way.

...

Kubernetes and GKE Limitations

There are known limitations with a single Kubernetes cluster that seem beyond our needs, however should be considered for proper design and future growth.

[Learn more about Kubernetes limits](#)

[Learn more about GKE design limitations](#)

Kubernetes Limitations

Quantity	Limit per namespace		Limit per cluster
Nodes	n/a	5000	
Namespaces	n/a	10000	
Pods	3000	150000	
Pods per node	min(110, 10cores)		min(110, 10cores)
Services	5000	10000	
Pods per service	250	n/a	

Pod CIDR Ranges

Pods per Node	CIDR Range per Node
8	/28
9-16	/27
17-32	/26
33-64	/25
65-110	/24

Node CIDR Ranges

Nodes per Subnet	Subnet CIDR
4	/29
12	/28
28	/27
60	/26
124	/25
252	/24
508	/23
1,020	/22
2,044	/21
4,092 (GKE default)	/20
8,188	/19
16,777,212	/8

Service CIDR Ranges

Services per Subnet	Subnet CIDR
32	/27
64	/26
128	/25
256	/24
512	/23
1,024	/22
2,048	/21
4,096 (GKE default)	/20
8,192	/19
16,384	/18
32,768	/17
65,536	/16

Multi-Dimension Limitations on Kubernetes Resources

Pod Secondary IP Range	Pod IP Addresses	Nodes	Pods
/24	256	1	110
/23	512	2	220
/22	1,024	4	440
/21	2,048	8	880

/20	4,096	16	1,760	
/19	8,192	32	3,520	
/18	16,384	64	7,040	
/17	32,768	128	14,080	
/16	65,536	256	28,160	
/15	131,072	512	56,320	
/14 (GKE default)	262,144		1,024	112,640
/13	524,288	2,048	225,280	
/12	1,048,576	4,096	450,560	
/11	2,097,152	8,192	901,120	

Google Best Practices for Optimizing IP Allocation

Learn more about flexible pod CIDR

By default, GKE configures nodes to run no more than 110 Pods. Kubernetes assigns each node a range of IP addresses, a CIDR block, so that each Pod can have a unique IP address. The size of the CIDR block corresponds to the maximum number of Pods per node.

With the default maximum of 110 Pods per node, Kubernetes assigns a /24 CIDR block (256 addresses) to each of the nodes. By having approximately twice as many available IP addresses as the number of pods that can be created on a node, Kubernetes is able to mitigate IP address reuse as Pods are added to and removed from a node.

Although having 110 Pods per node is a hard limit, you can reduce the number of Pods per node. If you reduce the number from the default value, Kubernetes assigns the node a correspondingly smaller CIDR block. The block always contains at least twice as many addresses as the maximum number of Pods per node. The table below lists the size of the CIDR block that Google Kubernetes Engine assigns to each node based on the maximum Pods per node.

When configuring the maximum number of Pods per node, you are indirectly configuring how much IP address space is required by each cluster node. For example, if you set the maximum Pods per node to 30 then, per the table above, a /26 CIDR range is used, and each Node is assigned 64 IP addresses. If you do not configure the maximum number of Pods per node, a /24 CIDR range is used, and each node is assigned 256 IP addresses.

Reducing the maximum number of Pods per node allows the cluster to have more nodes, since each node requires a smaller part of the total IP address space. Alternatively, you could support the same number of nodes in the cluster by specifying a smaller IP address space for Pods at cluster creation time.

Reducing the maximum number of Pods per node also lets you create smaller clusters that require fewer IP addresses. For example, with eight Pods per node, each node is granted a /28 CIDR. These Pod IP address ranges plus the user defined subnet and secondary ranges determine the number of IP addresses required to create a cluster successfully.

You can configure the maximum number of Pods per node at cluster creation time and at node pool creation time.

You can configure the maximum number of Pods per node when creating a cluster or when creating

a node pool. You cannot change this setting after the cluster or node pool is created.

You can set the size of the Pod address range when creating a cluster with either gcloud or Google Cloud Console.

You can also specify the maximum number of Pods per node when creating a node pool in an existing cluster. Creating a new node pool lets you optimize IP address allocation, even in existing clusters where default-max-pods-per-node wasn't configured at the cluster level.

This value overrides the default-max-pods-per-node option which is applied at the cluster-level. If you omit the max-pods-per-node option when creating a node pool, the cluster-level default configuration is used.

SECTION: customer-success/demo-systems/infrastructure/networking.md

Global CIDR Ranges

Each of the region/zone CIDR ranges exist in the 10.128.0.0/9 CIDR range. We have partitioned the /9 CIDR range into /12 CIDR ranges for each of the regions/zones.

CIDR	IP Range	(GitLab) Assigned To
-----	-----	-----
10.128.0.0 /12	10.128.0.0 - 10.143.255.255	demosys-saas-us-vpc (GCP us-central1)
10.144.0.0 /12	10.144.0.0 - 10.159.255.255	demosys-saas-eu-vpc (GCP eu-west1)
10.160.0.0 /12	10.160.0.0 - 10.175.255.255	demosys-saas-asia-vpc (GCP asia-southeast1)
10.176.0.0 /12	10.176.0.0 - 10.191.255.255	(Reserved for GCP expansion)
10.192.0.0 /16	10.192.0.0 - 10.192.255.255	demosys-mgmt-us-vpc (GCP us-central1)
10.193.0.0 /16	10.193.0.0 - 10.193.255.255	demosys-mgmt-eu-vpc (GCP eu-west1)
10.194.0.0 /16	10.194.0.0 - 10.194.255.255	demosys-mgmt-asia-vpc (GCP asia-southeast1)
10.195.0.0 /16	10.195.0.0 - 10.195.255.255	(reserved for future mgmt)
10.196.0.0 /16	10.196.0.0 - 10.196.255.255	(reserved for future mgmt)
10.197.0.0 /16	10.197.0.0 - 10.197.255.255	(reserved for future mgmt)
10.198.0.0 /16	10.198.0.0 - 10.198.255.255	(reserved for future mgmt)
10.199.0.0 /16	10.199.0.0 - 10.199.255.255	demosys-mgmt-global-vpc (GCP)

```
| 10.200.0.0 /13 | 10.200.0.0 - 10.207.255.255 | (Reserved for AWS mgmt)
|
| 10.208.0.0 /12    | 10.208.0.0 - 10.223.255.255 | (Reserved for AWS us-east-1)
|
| 10.224.0.0 /12    | 10.224.0.0 - 10.239.255.255 | (Reserved for AWS expansion)
|
| 10.240.0.0 /12    | 10.240.0.0 - 10.255.255.255 | (Reserved for AWS expansion)
|
```

{{% panel header="GitLab Implementation" header-bg="info" %}}

We like human recognizable infrastructure design elements and have allocated all of the 10.1xx. address space to GCP and all of the 10.2xx. IP space to AWS. This allow us to easily recognize which cloud provider console we should access to troubleshoot an issue. Feel free to adopt this idea for multi-cloud implementations, however it's not required.

Since we don't need as large of an IP range for management infrastructure, we partitioned the 10.192.0.0/12 CIDR range into two /13 ranges which allowed us to cleanly partition the IP space for our GCP and AWS management infrastructure. We further partitioned the /13 for GCP into multiple /16 ranges to allow us to divide our management infrastructure cleanly into multiple regions for redundancy.

{{% /panel %}}

How We Allocated Subnets

The standard subnet range for GCP VPC auto subnet mode is 10.128.0.0/9. This leaves the 10.0.0.0/9 range unused which is normally used for corporate networks (does not apply to GitLab). Our instincts would be to avoid conflicts with auto-created subnets and stick to using networks in the 10.0.0.0/9 range.

Since each VPC network has its own subnets, we can use the entire 10.0.0.0/8 range without issues as long as we don't use "auto subnet mode". In the spirit of an open source design that applies to many companies, we will stick to the 10.128.0.0/9 subnet range to avoid potential conflicts with corporate networks.

Global vs Regional Routing Tables

When configuring your VPC networks and routing tables, you can choose to create a global routing table with a 10.128.0.0/9 and subnets for each region/zone that can communicate with each other, or configure each region/zone to operate independently with a /12 routing table. Keep in mind that usage of terminology varies by cloud provider. For example, a GCP VPC is a global resource and an AWS VPC has a lot of options that are different in regards to cross-region network design.

{{% panel header="GitLab Implementation" header-bg="info" %}}

We keep each region isolated by creating a separate VPC Network for each region so there is no chance of cross-region contamination if something goes wrong. Although there are trade-offs with ease-of-administration that we solve with infrastructure-as-code tools, we prefer it this way to allow us to build/destroy/rebuild an entire region without worrying about whether the top-level resources are affected.

{{% /panel %}}

High-Availability with Multiple Availability Zones per Region

If you are designing a high-availability/fault-tolerant environment within a region and want to use multiple availability zones, you can choose to use a /12 CIDR range per zone (recommended for simplicity), or partition a /12 CIDR range into a /13 (2 zones) or /14 (3-4 zones). The documentation assumes that you're using a /12 per zone.

{{% panel header="GitLab Implementation" header-bg="info" %}}

We like human recognizable infrastructure design elements and have allocated all of the 10.1xx. address space to GCP and all of the 10.2xx. IP space to AWS. This allow us to easily recognize which cloud provider console we should access to troubleshoot an issue. Feel free to adopt this idea for multi-cloud implementations, however it's not required.

Since our demo environment is replicated in multiple regions, we don't think multi-zone redundancy within a region is necessary (or worth the compute cost). If we have a problem in a region, we'll redirect users to another region.

{{% /panel %}}

Region/Zone Subnet Ranges

The /12 CIDR for each region/zone includes 16 /16 CIDR ranges. In other words, there are 16 available values for the second octet of the IP address which provides 65,536 IPs per subnet. We have assigned an alpha character a through p to represent the 16 available values.

Subnet CIDR	Description
10.a.10.0 /24	demosys-{regionAbbrev}-mgmt-red-subnet
10.a.20.0 /24	demosys-{regionAbbrev}-mgmt-yellow-subnet
10.a.30.0 /24	demosys-{regionAbbrev}-app-red-subnet
10.a.40.0 /24	demosys-{regionAbbrev}-app-yellow-subnet
10.b.0.0 /16	(reserved for future)
10.c.0.0 /20	gke-{regionAbbrev}-gl-shared-runners
10.c.16.0 /20	gke-{regionAbbrev}-gl-dedicated-runners
10.c.32.0 /20	(reserved for future service)
10.c.48.0 /20	(reserved for future service)
10.c.64.0 /20	(reserved for future service)
10.c.80.0 /20	(reserved for future service)
10.c.96.0 /20	(reserved for future service)
10.c.112.0 /20	(reserved for future service)
10.c.128.0 /20	(reserved for future service)
10.c.144.0 /20	(reserved for future service)
10.c.160.0 /20	(reserved for future service)
10.c.176.0 /20	(reserved for future service)
10.c.192.0 /20	(reserved for future service)
10.c.208.0 /20	(reserved for future service)
10.c.224.0 /20	(reserved for future service)
10.c.240.0 /20	(reserved for future service)
10.d.0.0 /16	gke-{regionAbbrev}-gl-omnibus-instance
10.e.0.0 /16	(reserved for future)
10.f.0.0 /16	(reserved for future)

10.g.0.0 /16	(reserved for future)	
10.h.0.0 /16	(reserved for future)	
10.i.0.0 /16	(reserved for future)	
10.j.0.0 /16	(reserved for future)	
10.k.0.0 /16	(reserved for future)	
10.l.0.0 /16	(reserved for future)	
10.m.0.0 /16	(255) demosys-{regionAbbrev}-k8s-sandbox-{rsvpID}	
10.n.0.0 /16	(255) demosys-{regionAbbrev}-k8s-sandbox-{rsvpID}	
10.o.0.0 /16	(255) demosys-{regionAbbrev}-k8s-sandbox-{rsvpID}	
10.p.0.0 /16	(255) demosys-{regionAbbrev}-k8s-sandbox-{rsvpID}	

{{% panel header="GitLab Implementation" header-bg="info" %}}

This is where the "it depends" scenarios start with network design. You can allocate this IP space however you'd like including subnet partitioning or combining. All of the documentation is modeled after how we've implemented it at GitLab based on our use cases.

{{% /panel %}}

Demo Systems Management

demosys-mgmt-vpc

VPC Name	Region	CIDR	
-----	-----	-----	
demosys-mgmt-us-vpc	(gcp) us-central1-c	10.192.0.0 /16	

Subnets

Subnet CIDR	Subnet Name	
-----	-----	
10.192.10.0 /24	demosys-mgmt-us-mgmt-red-subnet	
10.192.20.0 /24	demosys-mgmt-us-mgmt-yellow-subnet	

Terraform Configuration

Learn more in our [Terraform US region configuration](#) and our [Terraform region VPC module](#).

console

```
cd ~/Sites/gitlab-com/customer-success/demo-systems/infrastructure/demosys-terraform
```

```
cd environments/demosys-mgmt/us
```

```
terraform init
```

```
terraform plan
```

```
terraform apply
```

US Demo Resources

VPC Name	Region	CIDR	
-----	-----	-----	
demosys-saas-us-vpc	(gcp) us-central1-c	10.128.0.0 /12	

Subnets

Subnet CIDR	Subnet Name	
10.128.30.0 /24	demosys-saas-us-app-red-subnet	
10.128.40.0 /24	demosys-saas-us-app-yellow-subnet	
10.130.0.0 /20	gke-us-k8s-gl-shared-runners-subnet-	
10.130.16.0 /20	gke-us-k8s-gl-dedicated-runners-subnet-	
10.131.0.0 /16	gke-us-k8s-gl-omnibus-instance-subnet-	
10.140.0.0 /16	(255) demosys-us-k8s-sandbox-{rsvplD}	
10.141.0.0 /16	(255) demosys-us-k8s-sandbox-{rsvplD}	
10.142.0.0 /16	(255) demosys-us-k8s-sandbox-{rsvplD}	
10.143.0.0 /16	(255) demosys-us-k8s-sandbox-{rsvplD}	

Terraform Configuration

Learn more in our [Terraform US region configuration](#) and our [Terraform region VPC module](#).

console

```
cd ~/Sites/gitlab-com/customer-success/demo-systems/infrastructure/demosys-terraform
```

```
cd environments/demosys-saas/us
```

```
terraform init
```

```
terraform plan
```

```
terraform apply
```

EU Demo Resources

demosys-eu-vpc

TODO

Asia Demo Resources

demosys-asia-vpc

TODO

Future Planning Considerations

We should...

1. have IP range capacity for 10x-100x our current requirements if it's a scalable service.
1. have fewer subnets with larger CIDR ranges to reduce complexity and plan for growth.
1. have separate subnets for each service that poses security or sandbox risk. For example, we should have a separate subnet for Kubernetes pods that users create or are using for sandbox purposes.
1. avoid re-using subnets across multiple VPCs to prevent human confusion and internal IP address overlapping. Keep in mind that Google Cloud uses internal DNS to avoid the issue with overlapping subnets, however this is a vendor-specific technology and is not something we

should depend on with our vendor agnostic design.

SECTION: customer-success/demo-systems/tutorials/_index.md

If you have issues, please post in the demo-systems channel on Slack for assistance.

Getting Started

Getting Started

- Configuring GitLab with group-level Kubernetes cluster

<!--

Advanced Use Cases

Advanced Use Cases

Using GitLab Pages

-->

<!--

CI/CD and Auto DevOps

CI/CD and Auto DevOps

Using Auto DevOps

Configuring your own runner

-->

Integrations

Integrations

- Create a Jenkins pipeline
- Using the JIRA integration

<!--

Contributed Tutorials

Contributed Tutorials

No tutorials available

-->

SECTION: customer-success/demo-systems/tutorials/getting-started/_index.md

Configuring GitLab with group-level Kubernetes cluster

Configuring GitLab with group-level Kubernetes cluster

After your GitLab Demo Cloud account has been created, your runners and CI jobs will use the pre-configured instance-level cluster by default. The instance-level cluster is designed to be transparent behind the scenes and does not support customization or administration and reporting capabilities as a demo systems user.

You can request your own Kubernetes cluster that is provisioned in Google Cloud's Google Kubernetes Engine (GKE) service by the demo systems team. By having your own cluster, you have full administrative access to the cluster for troubleshooting CI jobs and pods that are experiencing issues.

This tutorial shows you how to connect your cluster to your GitLab group after you are notified that your cluster is ready. You will access the Google Cloud Platform (GCP) console to see your cluster, use the Google Cloud Shell to run kubectl commands on your cluster, and access the GitLab Demo Cloud Omnibus instance to configure your pre-created group with a Kubernetes cluster. After your cluster has been added, we will install Helm, Prometheus and install your own GitLab Runner.

SECTION: customer-success/demo-systems/tutorials/getting-started/configuring-group-cluster.md

Overview

After your GitLab Demo Cloud account has been created, your runners and CI jobs will use the pre-configured instance-level cluster by default. The instance-level cluster is designed to be transparent behind the scenes and does not support customization or administration and reporting capabilities as a demo systems user.

You can create your own Kubernetes cluster using in your own GCP project that you can provision using the GitLab Sandbox Cloud. By having your own cluster, you have full administrative access to the cluster for troubleshooting CI jobs and pods that are experiencing issues.

This tutorial shows you how to create a cluster and connect it to your GitLab group. You will access the Google Cloud Platform (GCP) console to see your cluster, use the Google Cloud Shell to run kubectl commands on your cluster, and access the GitLab Demo Cloud Omnibus instance to configure your pre-created group with a Kubernetes cluster.

For a more intuitive UI for managing your cluster resources, we recommend using K9s.

Deprecation Notice

This method of connecting your cluster was deprecated in GitLab v14.5. This tutorial is preserved for educational purposes. See the Kubernetes Agent documentation for the new method for connecting clusters to GitLab.

Pre-Requisites

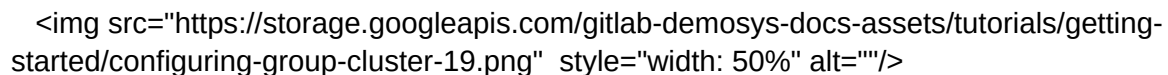
- Creating and accessing your Demo Systems account
- Create a GCP project using the GitLab Sandbox Cloud

Step-by-Step Instructions

Task 1. Create a Kubernetes cluster using the Google Cloud Platform (GCP) Console (Web UI)

For additional assistance with accessing your GCP project, see the GitLab Sandbox Cloud handbook instructions.

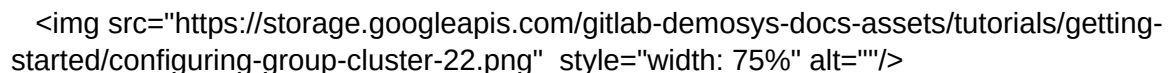
1. Open a new tab or window in your web browser.
2. Visit <https://console.cloud.google.com>.
3. If you're not already signed in, please sign in with your GitLab email account.
4. In the blue top navigation bar, locate the dropdown menu to the right of the Google Cloud Platform title. These are the projects (authentication realms) that you have access to. Make sure your project (ex. jmartin-xxxxxxx) is selected.
5. In the blue top navigation bar, locate and click on the hamburger menu icon to the left of the Google Cloud Platform title. These are all of the services that are available in the Google Cloud Platform. In the Compute section, click on Kubernetes Engine or Kubernetes Engine > Clusters.

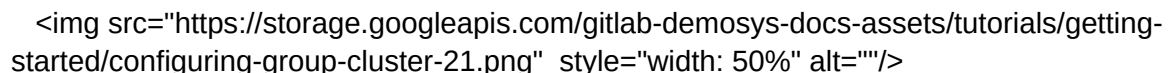


6. On the Kubernetes clusters page, click on ENABLE to initialize the Google API service. This action might take a few moments.



7. In the new page, click on CREATE then on CONFIGURE to the right of GKE Standard.





8. On the left-hand menu, click on the following sections to configure your cluster:

- Cluster basics: specify your cluster name (ex. jmartin-cluster) and make sure to select Zonal as 'Location Type'.



started/configuring-group-cluster-23.png" style="width: 75%" alt=""/>/>

- Node Pools -> default-pool: specify 1 (one) for the 'Number of nodes'

/>

- Node Pools -> default-pool -> Nodes: select e2-standard-2 for the 'Machine type' (recommended)

/>

9. Click on CREATE on the bottom navigation bar. When your cluster appears, click on the name of your cluster.

10. You are now at the dashboard for your cluster. It is recommended to bookmark this page for future access.

Task 2. Obtain the Cluster CA Certificate

1. Create a new file in your preferred text editor so we can copy and paste values into it for temporary reference.

1. Copy and paste your cluster name (at the top of the cluster dashboard) into the text file (Ex. jmartin-cluster).

1. In your browser on the cluster dashboard in the GCP console, locate and click on the Show credentials link next to the Endpoint (IP address value).

1. In the Cluster credentials pop-up modal window, copy and paste each of the values to your text editor file. For the value for the Cluster CA certificate, be sure to include the hyphen lines for BEGIN CERTIFICATE and END CERTIFICATE.

1. Click the Close button in the bottom right corner.

Task 3. Connect to Google Cloud Shell for configuring gitlab-admin service account

> These instructions have been customized from the GitLab documentation to show the specific instructions for the demo systems infrastructure.

1. On your cluster dashboard in the GCP console, locate and click on the Connect button in the top page navigation (below the search bar in the blue top navigation).

1. In the Connect to the cluster pop-up modal, click the Run in Cloud Shell button.

If you have experience with kubectl and gcloud command-line tools, you can optionally use the command shown. If you want to use them locally, you need to install them from here (kubectl, gcloud).

1. At the bottom of your browser window, a terminal window will appear. It may take a few moments for the Cloud Shell to finish provisioning.

The Cloud Shell is a small container that Google provides for you to have terminal access in your browser without needing to configure your local machine.

1. When the Cloud Shell terminal appears, the first line will be pre-populated with a command to obtain the credentials for your cluster using role-based access control (RBAC). Simply press the ENTER key.

```
console
jmartin@cloudshell:~ (jmartin-xxxxxxx)$ gcloud container clusters get-credentials jmartin-
cluster --zone us-central1-c --project jmartin-xxxxxxx
```

text

Fetching cluster endpoint and auth data.

kubeconfig entry generated for demosys-us-jmartin-cluster.

If you do not see the output shown in the example above, it is likely that your GCP account has permissions issues and you should contact demo-systems team for assistance.

1. Use the following command to get the API URL and copy/paste the outputted URL to your text file.

```
console
kubectl cluster-info | grep 'Kubernetes master' | awk '/http/ {print $NF}'
```

For recent versions of Kubernetes:

```
console
kubectl cluster-info | grep 'Kubernetes control plane' | awk '/http/ {print $NF}'
```

text

<https://35.239.222.203>

This will return an IP address that is the same as the endpoint IP that you can see in the GCP Console Web UI with an https:// prefix. Keep in mind that this URL does not provide a web UI for your cluster and is only used for API purposes.

1. Use the following command to create the service account.

```
console
kubectl create sa gitlab-admin -n kube-system
kubectl create clusterrolebinding gitlab-admin --serviceaccount=kube-system:gitlab-admin
--clusterrole=cluster-admin
```

You will receive a confirmation.

```
text
serviceaccount "gitlab-admin" created
clusterrolebinding "gitlab-admin" created
```

1. Use the following command to get the API token for the gitlab-admin service account and copy/paste the value of the token to your text editor.

```
console
kubectl -n kube-system describe secret $(kubectl -n kube-system get secret | grep gitlab-admin | awk '{print $1}')
```

```
text
Name:      gitlab-admin-token-b5zv4
Namespace: kube-system
Labels:    <none>
Annotations: kubernetes.io/service-account.name=gitlab-admin
             kubernetes.io/service-account.uid=bcfe66ac-39be-11e8-97e8-026dce96b6e8
```

Type: kubernetes.io/service-account-token

Data

====

namespace: 11 bytes

token: eyJhbGciOiJSUzI1NiIsImtpZCI6ImFCSFZicS1DbUpYbkZDNnRBcnc0TzhYeDNXbnBJMWoxck9UkhOYVUifQ.eyJpc3MiOiJrdWJlcm5ldGVzL3NlcnZpY2VhY2NvdW50Iiwia3ViZXJuZXRlcy5pby9zZXJ2aWN0dC9uYW1lc3BhY2UiOiJrdWJlLXN5

c3RlbSIsImt1YmVybmV0ZXMuaW8vc2VydmljZWFiY291bnQvc2VjcmV0Lm5hbWUiOiJnaXR5YWltYWRTaW1lbnRlc3RldGVzLmlvL3NlcnZpY2VhY2NvdW50L3NlcnZpY2UtYWNjb3VudC5uYW1lIjoZ2I0bGR1liwia3ViZXJuZXRlcy5pby9zZXJ

2aWNIYWNjb3VudC9zZXJ2aWNILWFjY291bnQudWkljoiOGZhODFkMzktZTM0NC00ZDQxLTkzMmEtZmNkliwic3Viljoic3lzdGVtOnNlcnZpY2VhY2NvdW50Omt1YmUtc3lzdGVtOmdpdGxhYi1hZG1pbiJ9.l0W-jWN9Ktvoi0z34y2jaTCcr9SiPH-vXpvXYMfeus7

T84mSqlztkTBW9S8V2mZBeQVbVMJERjUDrCyKwcvH1vwSJPNUYFJLCONDbevm1i9VgJz-ieCkQtI0FCgnCKZmJot76FEQpXjGrwRmZ0JQDxWtg680bLxEYqv7DZXcZzWNusdYOufziveiU9xE8b1XiprqeEj6Gy7BzFf7D5rgAGi6GNayrGWZ8YXYvi1dhEekG-pW8

MkZ8MTKopynxgIJ8ksvzCRSiwQ79H7mSR3TwTXoqjjzD3XOY2glACGt2tpLRJu3mTuUQWkrL69bfLFm

ca.crt: 1115 bytes

1. In your text editor, remove the line breaks in the token so that it is on a single line.

The line breaks appear as a space in the token when pasting into the GitLab UI and will result in a connection error with invalid credentials if you do not remove the line breaks.

1. You have successfully obtained the credentials needed to connect your cluster from GitLab.

Task 4. Accessing the GitLab Instance

See the demo systems handbook page instructions for accessing the GitLab Omnibus instance.

Task 6. Add Kubernetes Cluster to GitLab Group

It is imperative that you configure the cluster inside of your own group and not from the Admin section. If you make changes in the Admin section, this will cause breaking changes for the entire team. Please be mindful of the admin privileges that your account has for demo purposes.

1. In the left side bar for your group (ex. Jeff Martin), locate and click on Kubernetes.

1. On the Kubernetes page, you will see a list of clusters that are available. If you have not added a group cluster, this should only show instance clusters.

1. In the top right corner, click the Add Kubernetes cluster button.

1. On the Add a Kubernetes cluster integration page, click the Add existing cluster tab.

1. In the form fields, use the following values and copy and paste the appropriate values from your text editor file.

text

Kubernetes cluster name: (copy/paste - ex. jmartin-cluster)

Environment scope:

API URL: (copy/paste - ex. https://35.239.222.203)

CA Certificate: (copy/paste)

Service Token: (copy/paste)

RBAC-enabled cluster: checked

GitLab-managed cluster: checked

1. Click Add Kubernetes cluster.

Review

You have successfully configured a Kubernetes cluster with your group on GitLab.

You can now close the GCP console tab and continue using GitLab with Kubernetes integrated.

SECTION: customer-success/demo-systems/tutorials/integrations/_index.md

Create a Jenkins Pipeline

Create a Jenkins Pipeline

GitLab's Jenkins integration allows you to trigger a Jenkins build when you push code to a repository, or when a merge request is created. Additionally, it shows the pipeline status on merge requests widgets and on the project's home page.

This tutorial shows you how to create a project with a Jenkinsfile, configure a project on the Jenkins server, configure the GitLab Jenkins integration plugin, enable the integration on your GitLab project, and perform a commit to show how pipelines are integrated between GitLab and Jenkins.

SECTION: customer-success/demo-systems/tutorials/integrations/create-jenkins-pipeline.md

Overview

> Deprecation Notice: This tutorial is out-of-date and is only preserved for educational purposes. These steps no longer work on the GitLab Demo Systems.

GitLab's Jenkins integration allows you to trigger a Jenkins build when you push code to a repository, or when a merge request is created. Additionally, it shows the pipeline status on merge requests widgets and on the project's home page.

This tutorial shows you how to create a project with a Jenkinsfile, configure a project on the Jenkins server, configure the GitLab Jenkins integration plugin, enable the integration on your GitLab project, and perform a commit to show how pipelines are integrated between GitLab and Jenkins.

Step-by-Step Instructions

Task 1: Create a project for Jenkins CI pipeline integration

If you are just getting started with Jenkins, please follow the instructions in Option A to use our tutorial app that is a Ruby application with a pre-created Jenkinsfile.

If you're using your own existing project, please follow the instructions in Option B to add a Jenkinsfile to your application.

Option A: Create a new project using the tutorial app template

1. Visit <https://gitlab-core.us.gitlabdemo.cloud> and sign in with your Demo Cloud SSO credentials.

1. Create a new project and select the Create from template tab.

1. In the list of available projects, select the Instance tab.

1. Find Tutorial App - Jenkins Pipeline and click the Use template button.

1. In the new project form, use the following values:

- Project name: Tutorial App - Jenkins Pipeline
- Project URL: Groups > demosys-users/{MY-USERNAME}
- Project slug: tutorial-app-jenkins-pipeline
- Project description: (leave blank)
- Visibility Level: Private

1. After filling in the form, click the Create project button.

Option B: Add a Jenkinsfile to your existing project

Note: a Jenkinsfile will only be used if you create a pipeline job. You can skip this step if you want to show the integration with a freestyle job defined in Jenkins instead. Configuration for both options is included in this tutorial.

If you're using your own project, you will need to create a new file named Jenkinsfile (without a file extension) in the root level of your project where your README file is.

```
text
/app/
...
.gitignore
Jenkinsfile
README.md
```

As a starting point, you can use an example that shows a few basic Jenkins stages that will report status back to GitLab. Note: this Jenkinsfile does not actually perform a build or test, it just shows where the specific commands for build and test should go, using an echo.

```
json
pipeline {
  agent any
  stages {
    stage('build') {
      steps {
        echo 'Notify GitLab'
        updateGitlabCommitStatus name: 'build', state: 'pending'
        echo 'build step goes here'
        updateGitlabCommitStatus name: 'build', state: 'success'
      }
    }
  }
}
```

```

    }
    stage(test) {
        steps {
            echo 'Notify GitLab'
            updateGitlabCommitStatus name: 'test', state: 'pending'
            echo 'test step goes here'
            updateGitlabCommitStatus name: 'test', state: 'success'
        }
    }
}
}
}

```

There are additional examples provided in the Jenkins docs based on the language of your application.

Jenkinsfile Examples

You can reference the Jenkins docs for more detailed pipeline configurations, however that is outside the scope of this tutorial.

You can reference the open source Jenkins GitLab plugin documentation for more information on additional options.

Task 2. Connect to the Jenkins Server

1. Visit <https://gitlabdemo.com> and sign in with your Demo Cloud SSO credentials.
1. On the Dashboard, you will see all of the Demo Cloud resources that you have access to.
1. Locate the Jenkins Integration card (bordered section).
1. Click the blue Jenkins Dashboard button to open a new tab and access the Jenkins server.
1. You may or may not be prompted to sign in with your Demo Cloud SSO credentials depending on your browser session state. We have designed the integration to automatically sign in and take you to the dashboard.
- > In the Demo Cloud, we have configured the Jenkins server to use GitLab OAUTH for authentication. Keep in mind that this uses your Demo Cloud credentials and not your GitLab.com credentials.

1. You will be directed to the dashboard with pipeline jobs and folders.

Task 3. Create a Jenkins folder for your projects

Keep in mind that this is a shared environment and it is best practice create a folder with your username to keep your pipelines in and keep things tidy on the dashboard.

1. In the left sidebar, click New Item.
1. In the Enter an item name field, type in your GitLab Demo Cloud username (ex. jeffersonmartin).

> If you don't remember your username, switch back to the tab with the GitLab project. If you look in the browser URL, you can determine your username based on the path to your project.

1. Click the Folder type from the list of options.

1. Click the OK button in the bottom left corner.

1. When your folder is created, you will be redirected to a folder details page. You do not need to make any changes here.

Task 4. Create a Jenkins project (Freestyle Project or Pipeline)

1. In the top left of the page, click the Jenkins logo to navigate to the dashboard.

> It is understandable that the Jenkins UI can be a challenge to navigate. If you have any challenges, simply click on the Jenkins logo in the top left to go back to the dashboard and navigate from there.

1. Locate the folder for your username and click on the folder name hyperlink to navigate to the folder's dashboard.

1. In the left sidebar, click New item.

TIP: Create new items in your folder.

Keep in mind that since you're inside a folder, this item will be created in this folder and not at the top-level. It's easy to make this mistake so please be conscientious of this to avoid cluttering up the dashboard with uncategorized projects.

1. In the Enter an item name field, type in the hyphenated URL name (slug) of your GitLab project.

text

Example

tutorial-app-jenkins-pipeline

1. Choose either the Freestyle project or Pipeline type from the list of options.

1. Click the OK button in the bottom left corner.

1. You will be redirected to the project configuration page.

Task 5. Configure your Jenkins project

Option A - Freestyle Project (no Jenkinsfile needed for this)

1. If you are not on the project configuration page from the previous task, you can navigate here from the dashboard by clicking on your folder, then your project and clicking Configure link in the left sidebar.

1. In the General section, enable the checkbox for GitHub project.

> One of the cosmetic limitations of the integration is that the GitLab integration use the integration built for GitHub so there is some nomenclature overlap that use the same underlying Git integration.

1. Below the GitHub project checkbox, locate the Project url field copy/paste or type the URL of your GitLab project.

text

Example

<https://gitlab-core.us.gitlabdemo.cloud/demosys-users/jeffersonmartin/tutorial-app-jenkins-pipeline>

1. In the GitLab Connection field, select the GitLab Core US option from the dropdown menu if it's not already selected. This connection was configured by the system administrator earlier.

1. Scroll down to the Source Code Management section.

1. Select the Git radio checkbox.

1. In the expanded section, locate the Repository URL field and copy/paste the GitLab project URL (same URL as step 2).

text

Example

<https://gitlab-core.us.gitlabdemo.cloud/demosys-users/jeffersonmartin/tutorial-app-jenkins-pipeline>

> You may get an error message about connection error. This is expected behavior since there are no credentials selected. Simply proceed to the next step to select credentials from the dropdown menu.

1. Locate the Credentials dropdown menu and select the integjenkins option. Do not add new credentials in the demo environment.

> After you select the integjenkins option, the Jenkins server attempts to connect to the API of the GitLab instance to locate your repository URL. If successful, the error message will disappear automatically.

1. Click the Advanced button.

1. In the Name field, type in origin.

1. In the Refspec field, type in the following. This is directly from the documentation and is not customized for the demo environment.

text

+refs/heads/:refs/remotes/origin/ +refs/merge-requests//head:refs/remotes/origin/merge-requests/

1. In the Branch Specifier field, remove the /master field so that it is blank. This allows jobs to run for all branches (useful when testing a MR).

1. Scroll down to the Build Triggers section. Enable the checkbox for Build when a change is

pushed to GitLab. Leave all options at their default value.

1. Scroll down to the Build section. Click the Add build step dropdown menu and select the Set build status to "pending" on GitHub commit option.
1. Scroll down to the Post-build Actions section. Click the Add post-build action dropdown menu and select the Publish build status to GitLab option.
1. Click the Save button.

Option B - Pipeline project

1. If you are not on the project configuration page from the previous task, you can navigate here from the dashboard by clicking on your folder, then your project and clicking Configure link in the left sidebar.

1. In the General section, enable the checkbox for GitHub project.

> One of the cosmetic limitations of the integration is that the GitLab integration use the integration built for GitHub so there is some nomenclature overlap that use the same underlying Git integration.

1. Below the GitHub project checkbox, locate the Project url field copy/paste or type the URL of your GitLab project.

text

Example

<https://gitlab-core.us.gitlabdemo.cloud/demosys-users/jeffersonmartin/tutorial-app-jenkins-pipeline>

1. In the GitLab Connection field, select the GitLab Core US option from the dropdown menu if it's not already selected. This connection was configured by the system administrator earlier.

1. Scroll down to the Build Triggers section. Enable the checkbox for Build when a change is pushed to GitLab. Leave all options at their default value.

1. In the Pipeline section, use the Definition dropdown to choose Pipeline Script from SCM

1. For SCM, choose Git

1. In the expanded section, locate the Repository URL field and copy/paste the GitLab project URL (same URL as step 2).

text

Example

<https://gitlab-core.us.gitlabdemo.cloud/demosys-users/jeffersonmartin/tutorial-app-jenkins-pipeline>

> You may get an error message about connection error. This is expected behavior since there are no credentials selected. Simply proceed to the next step to select credentials from the dropdown menu.

1. Locate the Credentials dropdown menu and select the integjenkins option. Do not add new credentials in the demo environment.

> After you select the integjenkins option, the Jenkins server attempts to connect to the API of the GitLab instance to locate your repository URL. If successful, the error message will disappear automatically.

1. In the Branch Specifier field, replace the /master field with origin/\$gitlabSourceBranch

1. For Script Path, leave Jenkinsfile as is.

1. Keep Lightweight checkout checked

1. Click Save

Task 6. Generate an API Token for GitLab Integration

In the demo environment, each user generates their own API Token. In a production environment, you may want to create a service account-style user for the integration to simplify administration of API tokens if your security practices allow for that.

1. In the top left of the page, click the Jenkins logo to navigate to the dashboard.

1. In the left sidebar, click on People.

1. In the list of users, locate your account and click on your username.

1. In the left sidebar, click on Configure.

1. In the API Token section, click the Add new Token button.

1. In the form field with the Default name placeholder, type in GitLab integration and click Generate.

1. Create a new 1Password record or update your GitLab Demo Account record notes with the generated API token.

> If you navigate away from this page without copying the token, you will need to generate a new key since this token will not be visible again.

Task 7. Configure GitLab project to integrate with Jenkins

We will configure the integration in GitLab. In some of the plugin documentation, you will see configuration instructions for webhooks which are the previous generation of the integration. Although webhooks can be used, they do not provide the depth of integration that GitLab offers now.

If you still have your GitLab project in a different browser tab or window, you can skip to step 5.

1. Visit <https://gitlabdemo.com>.

1. On the Dashboard, locate the GitLab Omnibus (US) card (bordered section).

1. Click the blue My Group button to open a new tab and access your group on the GitLab instance.

1. In the list of projects on GitLab, click the title of the application you used earlier (Ex. Tutorial App - Jenkins Pipeline).

1. With the project open, verify that the Jenkinsfile exists in the root of the repository. You

do not need to make any changes.

> If the file does not exist, go back to Task 1 to create a Jenkinsfile or ensure that you're looking at the same GitLab project you started with earlier.

1. In the left sidebar, navigate to Settings > CI/CD.

1. On the CI/CD Settings page, locate Auto DevOps and click the Expand button.

1. Deselect the checkbox for Default to Auto DevOps pipeline. Click Save changes.

1. In the left sidebar, navigate to Settings > Integrations.

1. In the list of integrations, click on Jenkins CI.

1. Locate the Active checkbox and select to enable it.

1. In the Jenkins URL field, enter the Jenkins server base URL.

text

`https://jenkins.us.gitlabdemo.cloud`

1. In the Project name field, enter your folder name and Jenkins project name using {foldername}/{projectname} notation.

text

Example

`jeffersonmartin/tutorial-app-jenkins-pipeline`

1. In the Username field, enter your GitLab username (the same as your folder name).

1. In the Enter new password field, copy/paste the new API token that you just generated.

1. Click the Test settings and save changes button.

> If you get an error message, you may need to debug and see if your variables are correct.

This is a common area to experience a problem and some light troubleshooting usually fixes the issue.

Task 8. Create a Branch and MR and Then Change a File to Trigger a Commit Event

1. In the left sidebar, navigate to Repository > Branches.

1. Click the Create Branch button, give the branch a name, and click save.

1. Click the Create Merge Request button, scroll to the bottom of the page and click Submit Merge Request

1. Click Open in Web IDE

1. Locate the README.md file and click on the filename.

1. Make a change to the text (change a word, add a line, etc.)

1. Click the Commit changes button.

1. Navigate back to the Merge Request

Task 9. Review the Jenkins Pipeline in the GitLab UI

1. You should see the latest pipeline status in the merge request. This is valuable to

customers because they can now use the Jenkins pipeline status with the GitLab project merge check Pipelines must succeed to ensure that code is built and tested successfully prior to being merged to master.

1. Click the pipeline number to view the details.

1. In the pipeline details, if you have integrated a Jenkins freestyle job, you will see a single Jenkins job. If you integrated a pipeline job using the suggested Jenkinsfile, you should see two jobs - build and test. Right click on any job and select Open Link in New Tab.

1. The Jenkins UI will load and you'll see the build job that was executed in Jenkins.

1. In the left sidebar of the Jenkins UI, click Console Output.

1. Review the console output to see the details about the job that ran.

Review

You have successfully integrated your GitLab project with Jenkins CI.

There are several approaches to the authentication (shared vs per-user vs per-project), so please consider the best option for your environment when integrating Jenkins.

To learn more about the Jenkins integration, please see the official GitLab documentation.

You can also learn more in the [jenkinsci/gitlab-plugin](#) open source integration's README documentation on GitHub.

To learn more about using Jenkins with stages and multiple steps beyond our basic example, please see the Jenkins documentation.

SECTION: customer-success/education-services/_index.md

SECTION: customer-success/education-services/gitlab-education-services-support.md

Overview

This handbook will detail how to handle support inquiries for the Education Services team and the applicable systems that are used.

Support Systems in Use

The education services team has limited licenses for ZenDesk full agent- which gives users access to respond to customer tickets. To determine who has full agent access on this team,

please see here.

1. To access ZenDesk, launch the application through your Okta account.
2. On the left-hand side, click the Views button.
3. You will see 4 views available, My Assigned Tickets, Professional Services-Triage, Professional Services- Paid, and Professional Services- Free.
4. You will start in the My Assigned Tickets section, this is where any tickets you were assigned or have already responded to are, this tickets should be addressed first.
5. For new tickets, move to the Professional Services- Triage view, this is owned and triaged by L&D and Education Services.
6. To respond/resolve a ticket, assign the ticket to yourself, and complete the form data with your response in the Public reply field and click Submit as button- you will select the appropriate status based on whether the ticket will be Solved or Pending.
7. If there are Open tickets that are unassigned, with all regions or your region of preferred support, assign to yourself, complete the form data, and make a public reply if you can.
8. If you are not able to make a public reply, please ask for assistance from your colleagues to drive the ticket forward.
 - If the ticket is not related to Level Up or Education Services and is about general questions around GitLab or Sales, click the Apply Macro button, and select the following: General > Forms > Incorrect Form Used and click Submit
 - If the ticket is regarding a Sales Training or Certification, reach out to the Field Enablement team in the following slack channel: field-enablement-team and once you receive a resolution, respond to the ticket and close as Solved.
 - If the ticket is regarding a Learning and Development training or Certification such as Remote Foundations, TeamOps, etc, assign the ticket to the appropriate party listed here.
9. The Education Services team only handles the Professional Services- Triage and Professional Services- Paid views, the Learning and Development team owns the Professional Services- Free view.

Managing the LevelUp Inbox

In order to access Zapier, which is used for integrations and webhooks between systems (issuing badges, automated messaging), you need to have access to the Zapier 1Pass Vault. If you do not have access, please complete an Access Request.

All Zaps associated with the Education Services Team are found within the following folder in Zapier.

Credly

In order to access Credly, login to the site here. If you do not have access, please complete an Access Request.

GitLab Organization Profile

Credly Issuer Support Center

1. How to issue a badge

- Bulk Issuing Badges

- All of our badges are triggered via Webhook in Zapier. If you can see a passing result in the Certification Submissions, you can manually issue the badge, but please ping the eduserviceteam Slack channel so we can research the Integration issue.

1. Change the email on a badge

- If the badge is Accepted:

- You will have to click on the badge and click the Revoke button and then reissue to the new email.

- If the badge is Pending:

- You will be able to click on the badge and use the Replace button, you will see an edit icon next to the email field where you can change the email.

1. Pulling analytics from Credly

- We keep a track of certifications issued on a monthly basis and separated by audience on a Google Sheet located here. It is updated every month by Education Services Managers.

1. Creating a new badge template

- When creating a new badge, please be sure you engage the marketing design team to create the badge image to ensure it fits GitLab Branding Guidelines. You can use this issue template to open a request.

SECTION: customer-success/education-services/tech-stack/_index.md

SECTION: customer-success/education-services/tech-stack/articulate.md

The Tech Stack single source of truth is the Tech Stack YAML and contains more detail about this app.

{{% tech-stack "Articulate 360" %}}

Implementation

The Implementation of this system took place in March of 2020 and is currently used by the following departments: Education Services, Technical Marketing, Field Enablement, Learning and Development, and Recruiting.

System Diagram

The Articulate 360 tool is a SaaS app and is not integrated with other GitLab systems.

Data Model

The Data Model is not available and Articulate 360 is a closed system.

Integrations

The Articulate 360 tool is a SaaS app and is not integrated with other GitLab systems.

Key Reports / Dashboards

All Dashboards and Reports are a part of system itself. No separate Sisense reports are available or planned.

Support Guides and Step by Step Articles

The Articulate 360 Support page offers a docs website with full articles on processes and step by step guides on using the system.

SECTION: customer-success/education-services/tech-stack/credly.md

The Tech Stack single source of truth is the Tech Stack YAML and contains more detail about this app.

{{% tech-stack "Credly" %}}

Implementation

The Implementation of this system took place in May/June of 2022 as all digital badges were switched from our previous system Badgr to Credly.

System Diagram

The Credly digital credentialing system is a SaaS app and is integrated with the Thought Industries LMS.

```
mermaid
graph TD
    A[User starts completing content in Level Up] -->|Assignments/Submissions are graded| B(Level Up/Thought Industries registers completion and/or scores)
    B --> C{Completion based on score}
    C -->|Greater than 80%| D[Credly auto-issues the digital badge]
    C -->|Less than 80%| E[Level Up sends an email to the user to retake]
```

Data Model

The Data Model is as follows:

```
mermaid
graph TD
    A[Thought Industries collects user email and course completion/scores] -->|Assignments are graded| B(Course is completed successfully)
    B -->|80% or better completion| C[TI sends user email through native integration with Credly]
    C --> E[Credly issues digital credential to user via email]
```

Integrations

The Credly digital credentialing system is a SaaS app and is integrated with the Thought Industries LMS.

Key Reports / Dashboards

All Dashboards and Reports are a part of the system itself. No separate Sisense reports are available or planned.

Support Guides and Step by Step Articles

The Credly Support page offers a docs website with full articles on processes and step by step guides on using the system.

SECTION: customer-success/education-services/tech-stack/instruqt.md

The Tech Stack single source of truth is the Tech Stack YAML and contains more detail about this app.

{{% tech-stack "Instruqt" %}}

Implementation

The Instruqt implementation for GitLab is still underway and has not been full implemented.

System Diagram

The Instruqt Implementation is a SaaS app and is integrated with the Thought Industries LMS

mermaid

graph TD

A[Thought Industries LMS] -->|User name and email| B(Instruqt)

B -->|Virtual Labs completed| C[Completion data transferred back to Thought Industries]

Data Model

The Data Model is not available and Instruqt is a closed system.

Integrations

The Instruqt Implementation is a SaaS app and is integrated with the Thought Industries LMS

Key Reports / Dashboards

All Dashboards and Reports are a part of system itself. No separate Sisense reports are

available or planned.

Support Guides and Step by Step Articles

The Instruqt Support page offers a docs website with full articles on processes and step by step guides on using the system.

SECTION: customer-success/initiatives/_index.md

SECTION: customer-success/initiatives/communities-of-practice.md

What's a Community of Practice?

A Community of Practice (CoP) is a self-organized, cross-functional group of GitLab team members dedicated to a topic within GitLab or the broader DevSecOps space. The goal is to primarily to create thought leadership at GitLab, often in the form of content and other assets, with CoP members benefitting from sharing experiences with the topic. In turn, CoP's will build broader subject matter depth within our organization to advice our customers and our marketing and product teams.

The content and assets are aggregated into a single group to maximize discoverability. Each Community of Practice has a project which contains a Readme of links and an issue board for discussion.

Please note that even though Subject Matter Experts tend to participate in CoPs, that's technically a separate, more formal program.

Roles and Responsibilities

Role	Responsibility
---- ----	
Facilitator	Assembles the group, runs the meeting, and maintains the project.
Ideally 2 per CoP	
Contributor	Any team member who engages with the CoP group SME should try to contribute back - such as providing a deck or recording where the information gathered was utilized.

Guidelines

- CoP should consist of three or mor